



```
In [ ]: ''' Title of the Assignment: Text Analytics '''  
import nltk
```

```
In [28]: import pandas as pd
```

```
In [29]: import sklearn as sk
```

```
In [30]: import math
```

```
In [2]: nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to  
[nltk_data] C:\Users\avcoe\AppData\Roaming\nltk_data...  
[nltk_data] Package punkt is already up-to-date!
```

```
Out[2]: True
```

```
In [3]: from nltk.tokenize import sent_tokenize
```

```
In [4]: text="""Hello Mr. Smith, how are you doing today? The weather is  
great, and city is awesome. The sky is pinkish-blue. You shouldn't  
eat cardboard"""  
tokenize_text=sent_tokenize(text)
```

```
In [ ]:
```

```
In [5]: from nltk.tokenize import word_tokenize  
tokenized_word=word_tokenize(text)  
print(tokenized_word)
```

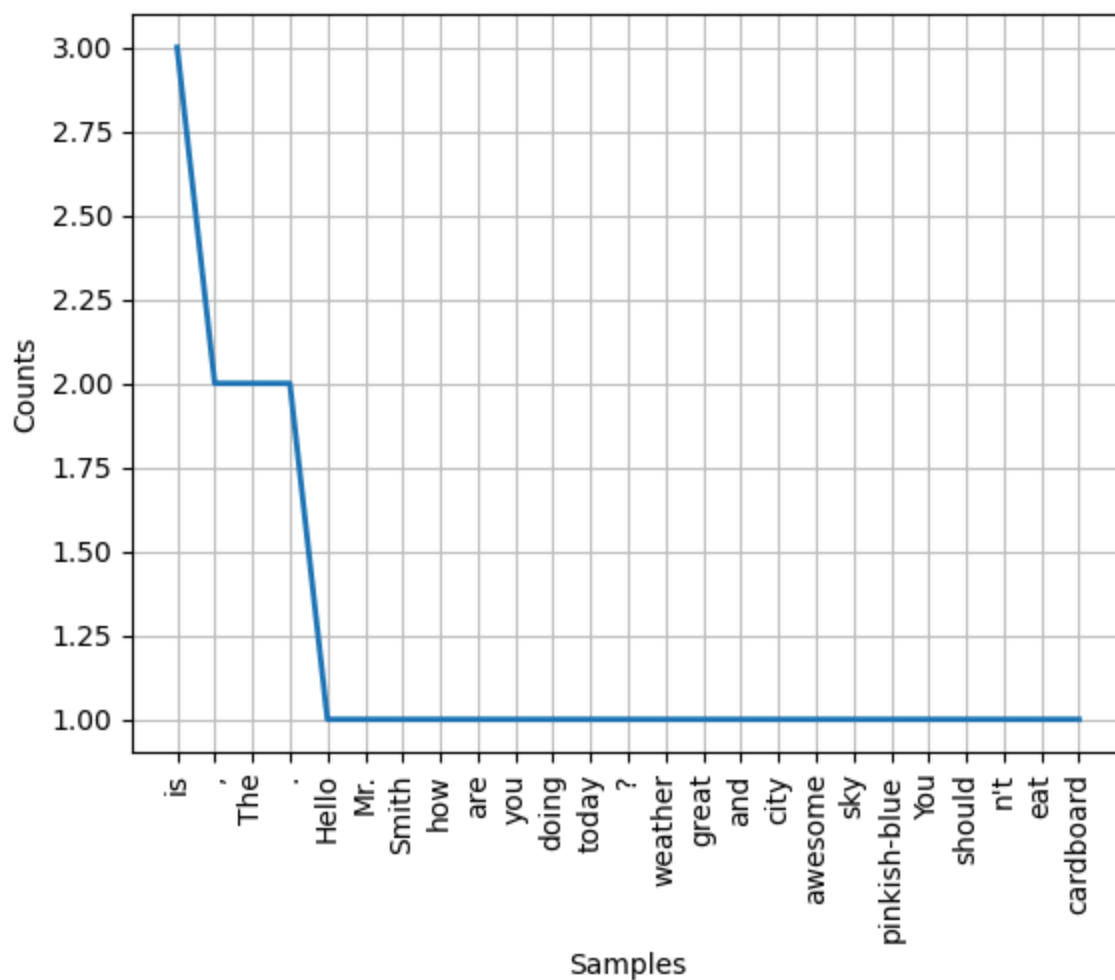
```
['Hello', 'Mr.', 'Smith', ',', 'how', 'are', 'you', 'doing', 'today', '?', 'Th  
e', 'weather', 'is', 'great', ',', 'and', 'city', 'is', 'awesome', '.', 'The',  
'sky', 'is', 'pinkish-blue', '.', 'You', 'should', "n't", 'eat', 'cardboard']
```

```
In [6]: from nltk.probability import FreqDist  
fdist = FreqDist(tokenized_word)  
print(fdist)
```

```
<FreqDist with 25 samples and 30 outcomes>
```

```
In [7]: import matplotlib.pyplot as plt
```

```
In [12]: fdist.plot(30,cumulative=False)  
plt.show()
```



```
In [9]: sent = "Albert Einstien was born was born in ulm , Germany"
```

```
In [10]: tokens=nltk.word_tokenize(sent)
print(tokens)
nltk.download('averaged_perceptron_tagger')
nltk.pos_tag(tokens)
```

```
['Albert', 'Einstien', 'was', 'born', 'was', 'born', 'in', 'ulm', ',', 'German', 'y']
```

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] C:\Users\avcoe\AppData\Roaming\nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
```

```
Out[10]: [('Albert', 'NNP'),
          ('Einstien', 'NNP'),
          ('was', 'VBD'),
          ('born', 'VBN'),
          ('was', 'VBD'),
          ('born', 'VBN'),
          ('in', 'IN'),
          ('ulm', 'NN'),
          (',', ','),
          ('Germany', 'NNP')]
```

```
In [16]: nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to  
[nltk_data] C:\Users\avcoe\AppData\Roaming\nltk_data...  
[nltk_data] Unzipping corpora\stopwords.zip.
```

```
Out[16]: True
```

```
In [17]: from nltk.corpus import stopwords  
stop_words=set(stopwords.words("english"))  
print(stop_words)
```

```
{'re', 'myself', 'we'd', 'i'd', 'was', 'y', 'doesn', 'whom', 'we', 'i'm', 'need  
n', 'by', 'you've', 've', 'above', 'isn't', 'he's', 'mustn't', 'why', 'won', 'm  
y', 'once', 'hadn't', 'were', 'down', 'itself', 'wasn't', 'ain', 'couldn't', 'b  
e', 'do', 'which', 'hasn't', 'has', 'where', 'he'll', 'in', 'shan't', 'agains  
t', 'those', 'not', 'it'd', 'they'd', 'd', 'no', 'didn', 'that'll', 'with', 'th  
ey're', 'she'll', 'for', 'theirs', 'she'd', 'so', 'mightn', 'most', 'only', 'yo  
u'll', 'does', 'you', 'her', 'now', 'i've', 'off', 'more', 'are', 'such', 'jus  
t', 'wouldn', 'aren', 'through', 'doesn't', 'as', 'who', 'mightn't', 'won't',  
'over', 'each', 'until', 'did', 'themselves', 'under', 'the', 'about', 'me', 's  
hould've', 'below', 'he', 'all', 'is', 'on', 'what', 'herself', 'between', 'nee  
dn't', 'will', 'further', 'these', 'how', 'o', 'any', 'after', 'both', 't', 'yo  
urs', 'himself', 'don't', 'nor', 'doing', 'she's', 'isn', 'a', 'i'll', 'out',  
'than', 'shouldn't', 'we're', 'didn't', 'him', 'ours', 'some', 'hadn', 'while',  
'to', 'you'd', 'yourself', 'wasn', 'this', 'then', 'or', 'if', 'hasn', 'your',  
'shouldn', 'its', 'when', 'am', 'because', 'should', 'yourselves', 'again', 'ar  
en't', 'had', 'he'd', 'm', 'mustn', 'from', 'ourselves', 'there', 'it's', 'yo  
u're', 'they'll', 'i', 'she', 'having', 'at', 'have', 'being', 'here', 'they',  
'up', 'it'll', 'of', 'we've', 'weren', 'into', 'and', 's', 'them', 'own', 'ma',  
'haven', 'few', 'we'll', 'been', 'that', 'but', 'too', 'hers', 'it', 'can', 'hi  
s', 'other', 'an', 'our', 'shan', 'they've', 'weren't', 'before', 'very', 'do  
n', 'during', 'same', 'll', 'couldn', 'wouldn't', 'haven't', 'their'}
```

```
In [19]: tokenized_sent = word_tokenize(text)
```

```
In [21]: filtered_sent=[]  
for w in tokenized_sent:  
    if w not in stop_words:  
        filtered_sent.append(w)
```

```
In [22]: print("Tokenized Sentence:",tokenized_sent)  
print("Filterd Sentence:",filtered_sent)
```

```
Tokenized Sentence: ['Hello', 'Mr.', 'Smith', ',', 'how', 'are', 'you', 'doin  
g', 'today', '?', 'The', 'weather', 'is', 'great', ',', 'and', 'city', 'is', 'a  
wesome', '.', 'The', 'sky', 'is', 'pinkish-blue', '.', 'You', 'should', 'n't',  
'eat', 'cardboard']  
Filterd Sentence: ['Hello', 'Mr.', 'Smith', ',', 'today', '?', 'The', 'weathe  
r', 'great', ',', 'city', 'awesome', '.', 'The', 'sky', 'pinkish-blue', '.', 'Y  
ou', 'n't', 'eat', 'cardboard']
```

```
In [23]: from nltk.stem import PorterStemmer
```

```
In [24]: ps = PorterStemmer()
stemmed_words=[]
for w in filtered_sent:
    stemmed_words.append(ps.stem(w))
print("Filtered Sentence:",filtered_sent)
print("Stemmed Sentence:",stemmed_words)
```

Filtered Sentence: ['Hello', 'Mr.', 'Smith', ',', 'today', '?', 'The', 'weather', 'great', ',', 'city', 'awesome', '.', 'The', 'sky', 'pinkish-blue', '.', 'You', 'n't', 'eat', 'cardboard']
 Stemmed Sentence: ['hello', 'mr.', 'smith', ',', 'today', '?', 'the', 'weather', 'great', ',', 'citi', 'awesom', '.', 'the', 'sky', 'pinkish-bl', '.', 'you', 'n't', 'eat', 'cardboard']

```
In [25]: from nltk.stem.wordnet import WordNetLemmatizer
lem = WordNetLemmatizer()
```

```
In [26]: from nltk.stem.porter import PorterStemmer
stem = PorterStemmer()
word = "flying"
```

```
In [27]: print("Lemmatized Word:",lem.lemmatize(word,"v"))
print("Stemmed Word:",stem.stem(word))
```

Lemmatized Word: fly
 Stemmed Word: fli

```
In [33]: first_sentence = "Data Science is the best job of the 21st century"
second_sentence = "machine learning is the key for data science"
# split so each word has its own string
first_sentence = first_sentence.split(" ")
second_sentence = second_sentence.split(" ")
# join them to remove common duplicate words
total = set(first_sentence).union(set(second_sentence))
print(total)
```

{'of', 'science', 'data', 'for', '21st', 'is', 'key', 'century', 'Data', 'best', 't', 'job', 'Science', 'the', 'learning', 'machine'}

```
In [34]: wordDictA = dict.fromkeys(total, 0)
wordDictB = dict.fromkeys(total, 0)
for word in first_sentence:
    wordDictA[word] += 1
for word in second_sentence:
    wordDictB[word] += 1
```

```
In [35]: pd.DataFrame([wordDictA, wordDictB])
```

```
Out[35]:
```

	of	science	data	for	21st	is	key	century	Data	best	job	Science	the
0	1	0	0	0	1	1	0	1	1	1	1	1	2
1	0	1	1	1	0	1	1	0	0	0	0	0	1

```
In [65]: def computeTF(wordDict, doc):
    tfDict = {}
    corpusCount = len(doc)

    for word, count in wordDict.items():
        tfDict[word] = count / float(corpusCount)

    return tfDict
```

```
In [66]: tfFirst = computeTF(wordDictA, first_sentence)
tfSecond = computeTF(wordDictB, second_sentence)
```

```
In [67]: # Converting to dataframe for visualization
import pandas as pd
tf = pd.DataFrame([tfFirst, tfSecond])
print(tf)
```

	of	science	data	for	21st	is	key	century	Data	best	job	\
0	0.1	0.000	0.000	0.000	0.1	0.100	0.000	0.1	0.1	0.1	0.1	
1	0.0	0.125	0.125	0.125	0.0	0.125	0.125	0.0	0.0	0.0	0.0	

	Science	the	learning	machine
0	0.1	0.200	0.000	0.000
1	0.0	0.125	0.125	0.125

```
In [38]: import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))
filtered_sentence = [w for w in wordDictA if w not in stop_words]
print(filtered_sentence)
```

```
['science', 'data', '21st', 'key', 'century', 'Data', 'best', 'job', 'Science',
'learning', 'machine']
```

```
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\avcoe\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

```
In [68]: import math

def computeIDF(docList):
    idfDict = {}
    N = len(docList)

    for doc in docList:
        for word in doc:
            if word not in idfDict:
                idfDict[word] = 1
            else:
                idfDict[word] += 1

    for word, val in idfDict.items():
        idfDict[word] = math.log10(N / float(val))
```

```
return idfDict
```

```
In [69]: docList = [first_sentence, second_sentence]
idfDict = computeIDF(docList)
```

```
In [70]: import pandas as pd

def computeTFIDF(tfBow, idfs):
    tfidf = {}

    for word, val in tfBow.items():
        tfidf[word] = val * idfs.get(word, 0)

    return tfidf
```

```
In [71]: tfidfFirst = computeTFIDF(tfFirst, idfs)
tfidfSecond = computeTFIDF(tfSecond, idfs)

tfidf = pd.DataFrame([tfidfFirst, tfidfSecond])

print(tfidf)
```

	of	science	data	for	21st	is	key	century	\
0	0.030103	0.000000	0.000000	0.000000	0.030103	0.0	0.000000	0.030103	
1	0.000000	0.037629	0.037629	0.037629	0.000000	0.0	0.037629	0.000000	

	Data	best	job	Science	the	learning	machine
0	0.030103	0.030103	0.030103	0.030103	-0.035218	0.000000	0.000000
1	0.000000	0.000000	0.000000	0.000000	-0.022011	0.037629	0.037629

```
In [2]: from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

# sentences (all lowercase)
firstV = "data science is the sexiest job of the 21st century"
secondV = "machine learning is the key for data science"

# vectorizer
vectorize = TfidfVectorizer()

# fit + transform
response = vectorize.fit_transform([firstV, secondV])

# convert to dataframe for readable output
df = pd.DataFrame(response.toarray(), columns=vectorize.get_feature_names_out())

print(response)
```

<Compressed Sparse Row sparse matrix of dtype 'float64'
with 17 stored elements and shape (2, 13)>

Coords	Values
(0, 2)	0.24342026926924518
(0, 10)	0.24342026926924518
(0, 4)	0.24342026926924518
(0, 12)	0.48684053853849035
(0, 11)	0.34211869506421816
(0, 5)	0.34211869506421816
(0, 9)	0.34211869506421816
(0, 0)	0.34211869506421816
(0, 1)	0.34211869506421816
(1, 2)	0.28986933576883284
(1, 10)	0.28986933576883284
(1, 4)	0.28986933576883284
(1, 12)	0.28986933576883284
(1, 8)	0.40740123733358447
(1, 7)	0.40740123733358447
(1, 6)	0.40740123733358447
(1, 3)	0.40740123733358447